

# Indian E-Commerce Sales Analysis

**Dataset:** Brazilian E-Commerce Public Dataset (Olist)

## Dataset Size:

Customers: 20,000

Orders: 20,000

Order Items: 20,000

Products: 19,616

## Project Overview

This project analyzes e-commerce sales data using SQL and Excel to identify revenue trends, top-performing products, category performance, regional sales distribution, and order fulfillment effectiveness. The objective was to generate business insights and visualize key performance indicators through an interactive dashboard.

## Tools Used

- MySQL Workbench
- Microsoft Excel
- SQL
- Data Visualization

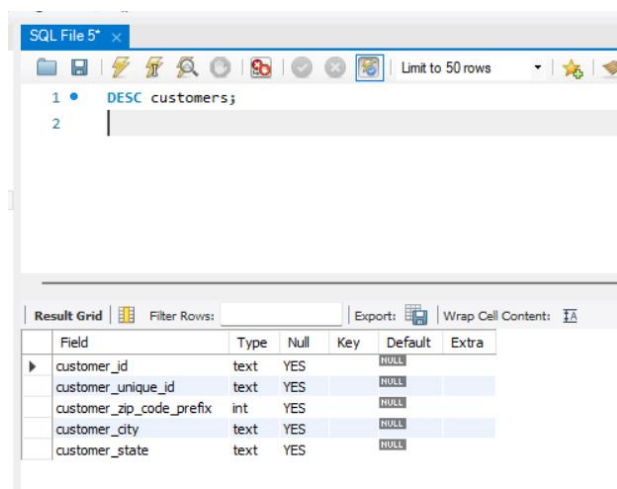
## Table Structure Analysis

DESC customers;

DESC orders;

DESC order\_items;

DESC products;



The screenshot shows the MySQL Workbench interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 50 rows' dropdown. The SQL editor contains the command 'DESC customers;'. The bottom 'Result Grid' displays the table structure for 'customers'.

| Field                    | Type | Null | Key | Default | Extra |
|--------------------------|------|------|-----|---------|-------|
| customer_id              | text | YES  |     | NULL    |       |
| customer_unique_id       | text | YES  |     | NULL    |       |
| customer_zip_code_prefix | int  | YES  |     | NULL    |       |
| customer_city            | text | YES  |     | NULL    |       |
| customer_state           | text | YES  |     | NULL    |       |

SQL File 5\* x

Limit to 50 rows

```
1 • DESC orders;
2
```

Result Grid

| Field                         | Type | Null | Key | Default | Extra |
|-------------------------------|------|------|-----|---------|-------|
| order_id                      | text | YES  |     | NULL    |       |
| customer_id                   | text | YES  |     | NULL    |       |
| order_status                  | text | YES  |     | NULL    |       |
| order_purchase_timestamp      | text | YES  |     | NULL    |       |
| order_approved_at             | text | YES  |     | NULL    |       |
| order_delivered_carrier_date  | text | YES  |     | NULL    |       |
| order_delivered_customer_date | text | YES  |     | NULL    |       |
| order_estimated_delivery_date | text | YES  |     | NULL    |       |

Result 30

SQL File 5\* x

Limit to 50 rows

```
1 • DESC order_items;
2
```

Result Grid

| Field               | Type   | Null | Key | Default | Extra |
|---------------------|--------|------|-----|---------|-------|
| order_id            | text   | YES  |     | NULL    |       |
| order_item_id       | int    | YES  |     | NULL    |       |
| product_id          | text   | YES  |     | NULL    |       |
| seller_id           | text   | YES  |     | NULL    |       |
| shipping_limit_date | text   | YES  |     | NULL    |       |
| price               | double | YES  |     | NULL    |       |
| freight_value       | double | YES  |     | NULL    |       |

Result 31

SQL File 5\* x

Limit to 50 rows

```
1 • DESC products;
2
```

Result Grid

| Field                      | Type | Null | Key | Default | Extra |
|----------------------------|------|------|-----|---------|-------|
| product_id                 | text | YES  |     | NULL    |       |
| product_category_name      | text | YES  |     | NULL    |       |
| product_name_lenght        | int  | YES  |     | NULL    |       |
| product_description_lenght | int  | YES  |     | NULL    |       |
| product_photos_qty         | int  | YES  |     | NULL    |       |
| product_weight_g           | int  | YES  |     | NULL    |       |
| product_length_cm          | int  | YES  |     | NULL    |       |
| product_height_cm          | int  | YES  |     | NULL    |       |
| product_width_cm           | int  | YES  |     | NULL    |       |

Result 34

## Dataset Validation

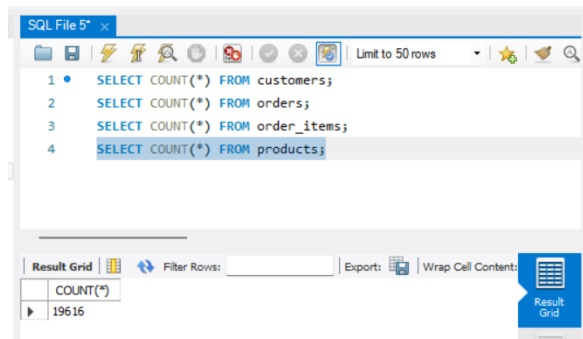
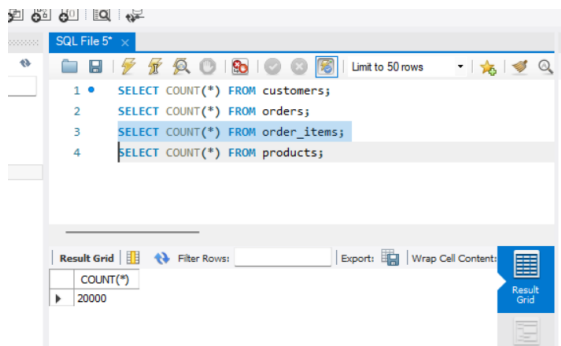
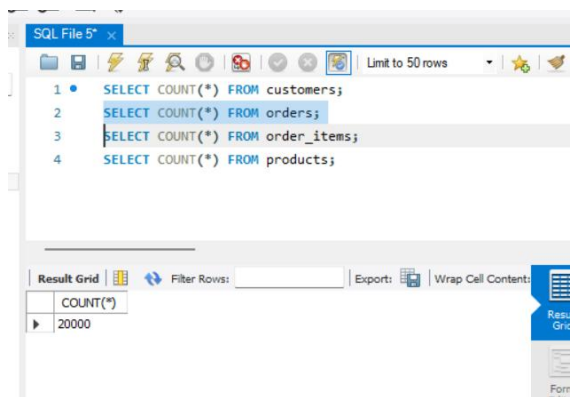
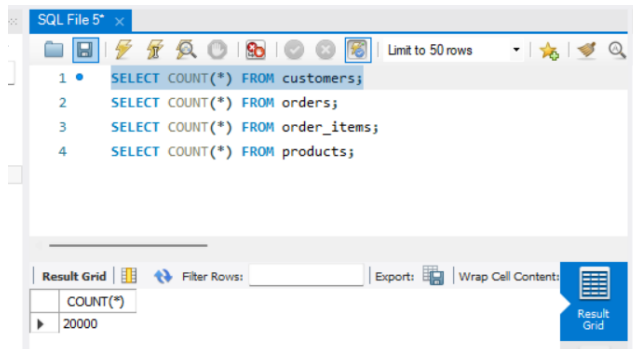
Verified the number of records available in each table before performing analysis:

```
SELECT COUNT(*) FROM customers;
```

SELECT COUNT(\*) FROM orders;

SELECT COUNT(\*) FROM order\_items;

SELECT COUNT(\*) FROM products;



## Database Schema

The project consists of four primary tables:

- customers – Stores customer location information.
- orders – Contains order details and purchase timestamps.
- order\_items – Contains product-level transaction details and pricing.
- products – Contains product category information.

Relationships:

customers → orders → order\_items → products

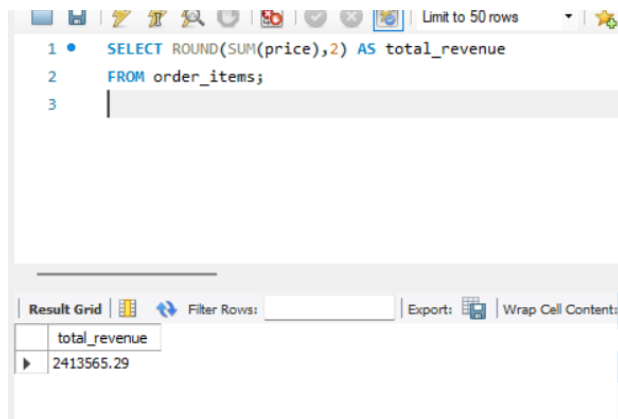
## KPI Analysis

### Query 1: Total Revenue

**Objective:** Calculate the total revenue generated from all orders in the dataset.

**SQL Query:** `SELECT ROUND(SUM(price),2) AS total_revenue FROM order_items;`

**Output:**



The screenshot shows a SQL query editor with the following query:

```
1 • SELECT ROUND(SUM(price),2) AS total_revenue
2 FROM order_items;
3
```

Below the query editor, the result grid is displayed with the following data:

| total_revenue |
|---------------|
| 2413565.29    |

**Business Insight:** The total revenue generated from the analyzed dataset was ₹241,356.29. This KPI provides an overall view of business performance and serves as the foundation for further sales analysis.

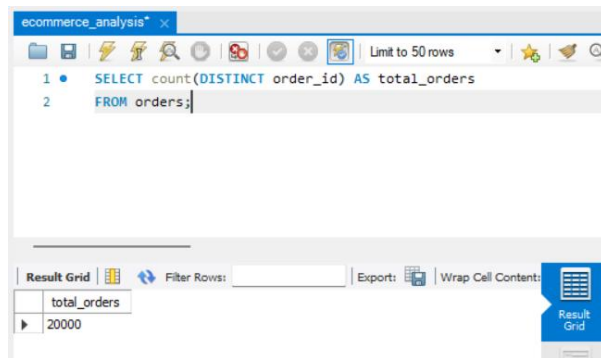
### Query 2: Total Orders

#### Objective

Determine the total number of orders available in the dataset.

**SQL Query:** `SELECT count(DISTINCT order_id) AS total_orders FROM orders;`

## Output:



The screenshot shows a SQL query editor with a query to count distinct orders. The query is: `SELECT count(DISTINCT order_id) AS total_orders FROM orders;`. The result grid shows a single row with the value 20000.

| total_orders |
|--------------|
| 20000        |

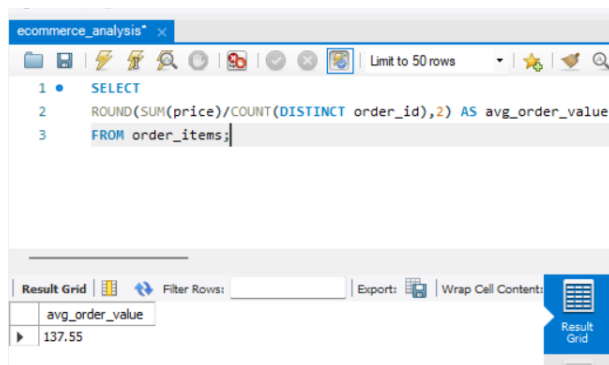
**Business Insight:** A total of 20,000 orders were analyzed. This metric helps measure transaction volume and provides context for revenue and customer activity analysis.

## Query 3: Average Order Value

**Objective:** Calculate the average revenue generated per order.

**SQL Query:** `SELECT ROUND(SUM(price)/COUNT(DISTINCT order_id),2) AS avg_order_value FROM order_items;`

## Output:



The screenshot shows a SQL query editor with a query to calculate the average order value. The query is: `SELECT ROUND(SUM(price)/COUNT(DISTINCT order_id),2) AS avg_order_value FROM order_items;`. The result grid shows a single row with the value 137.55.

| avg_order_value |
|-----------------|
| 137.55          |

**Business Insight:** The average order value was ₹137.55, indicating the average amount spent per transaction. This metric helps businesses understand customer purchasing behavior and evaluate sales performance.

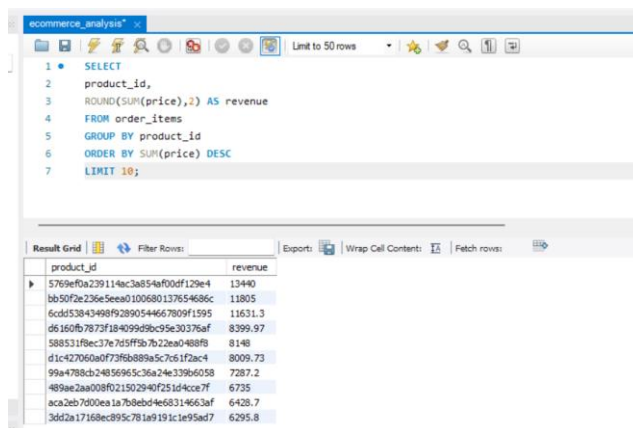
## Revenue Analysis

### Query 4: Top 10 Products by Revenue

**Objective:** Identify the highest revenue-generating products to understand which products contribute most to overall sales.

**SQL Query:** `SELECT product_id, ROUND(SUM(price),2) AS revenue FROM order_items GROUP BY product_id ORDER BY SUM(price) DESC LIMIT 10;`

## Output:



The screenshot shows a SQL query editor with the following query:

```
1 SELECT
2   product_id,
3   ROUND(SUM(price),2) AS revenue
4 FROM order_items
5 GROUP BY product_id
6 ORDER BY SUM(price) DESC
7 LIMIT 10;
```

The result grid displays the top 10 products by revenue:

| product_id                       | revenue |
|----------------------------------|---------|
| 5799ef0a2291144ec3a854a00df129e4 | 13440   |
| bb50f2e226e5eeao100680137654686c | 11805   |
| 6cdd53943498f92890544667809f1595 | 11631.3 |
| d6160fb7873f18409d9dc95e30376af  | 8399.97 |
| 588531f8ec37e7d5ff5b7b22ea0488fb | 8148    |
| d1c427060a0f72f6b889a5c7c51f2ac4 | 8009.73 |
| 99a4788cb24856965c36a24e339b6058 | 7287.2  |
| 489ae2aa008f021502940f251d4cce7f | 6735    |
| aca2eb7000ea1a7b8ebd4e68314663af | 6428.7  |
| 3dd2a17168ec895c781e9191c1e95ad7 | 6295.8  |

**Business Insight:** The top-performing product generated approximately ₹13,440 in revenue, indicating that a small group of products contributes significantly to overall business revenue.

## Query 5: Top Categories by Revenue

**Objective:** Identify product categories contributing the highest revenue.

### Tables Used:

- order\_items
- products

**SQL Concepts Used:** INNER JOIN, GROUP BY, SUM, ORDER BY

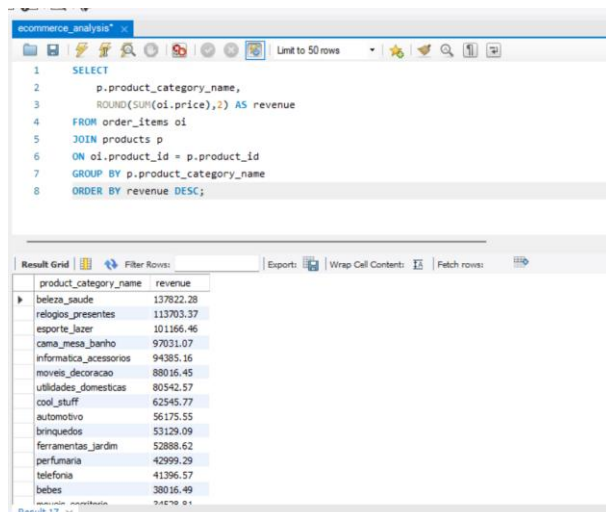
### SQL Query: SELECT

p.product\_category\_name,

ROUND(SUM(oi.price),2) AS revenue

FROM order\_items oi JOIN products p ON oi.product\_id = p.product\_id GROUP BY  
p.product\_category\_name ORDER BY revenue DESC;

## Output:



The screenshot shows a SQL query editor with the following query:

```
1 SELECT
2   p.product_category_name,
3   ROUND(SUM(oi.price),2) AS revenue
4 FROM order_items oi
5 JOIN products p
6 ON oi.product_id = p.product_id
7 GROUP BY p.product_category_name
8 ORDER BY revenue DESC;
```

The result grid displays the top product categories by revenue:

| product_category_name  | revenue   |
|------------------------|-----------|
| beleza_saude           | 137822.28 |
| relorios_presentes     | 113703.37 |
| esporte_lazer          | 101166.46 |
| cama_mesa_banho        | 97031.07  |
| informatica_acessorios | 94385.16  |
| moveis_decoracao       | 88016.45  |
| utilidades_domesticas  | 80542.57  |
| cool_stuff             | 62545.77  |
| automotivo             | 56175.55  |
| brinquedos             | 53129.09  |
| ferramentas_jardim     | 52888.62  |
| perfumaria             | 42999.29  |
| telefonia              | 41396.57  |
| bebes                  | 38016.49  |
| marcas_novidades       | 34478.81  |

**Business Insight:** Product category analysis revealed that a few categories contribute a significant portion of total revenue. These categories should be prioritized for inventory management, promotions, and marketing campaigns.

## Trend Analysis

### Query 6: Monthly Order Trend

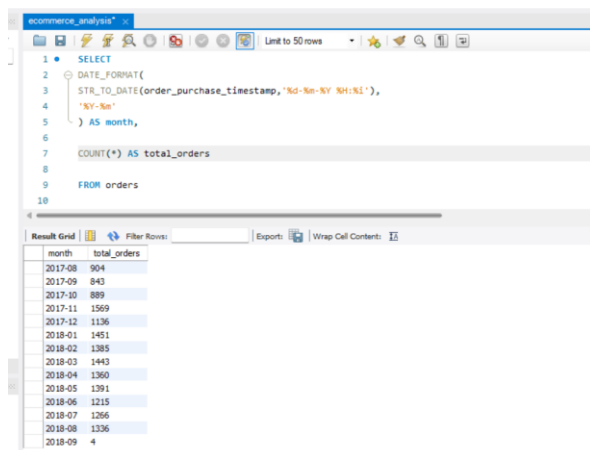
**Objective:** Analyze month-wise order volume to identify growth patterns and seasonal trends.

**SQL Concepts Used:** STR\_TO\_DATE, DATE\_FORMAT, GROUP BY, COUNT, ORDER BY

**SQL Query:**

```
SELECT DATE_FORMAT(  
STR_TO_DATE(order_purchase_timestamp,'%d-%m-%Y %H:%i'),  
'%Y-%m'  
) AS month, COUNT(*) AS total_orders FROM orders GROUP BY month ORDER BY month;
```

**Output:**



The screenshot shows a SQL query editor with the following query:

```
1 SELECT  
2 DATE_FORMAT(  
3 STR_TO_DATE(order_purchase_timestamp,'%d-%m-%Y %H:%i'),  
4 '%Y-%m'  
5 ) AS month,  
6  
7 COUNT(*) AS total_orders  
8  
9 FROM orders  
10
```

Below the query editor is a 'Result Grid' showing the output of the query. The grid has two columns: 'month' and 'total\_orders'. The data is as follows:

| month   | total_orders |
|---------|--------------|
| 2017-08 | 904          |
| 2017-09 | 843          |
| 2017-10 | 889          |
| 2017-11 | 1569         |
| 2017-12 | 1136         |
| 2018-01 | 1451         |
| 2018-02 | 1385         |
| 2018-03 | 1443         |
| 2018-04 | 1360         |
| 2018-05 | 1391         |
| 2018-06 | 1215         |
| 2018-07 | 1266         |
| 2018-08 | 1336         |
| 2018-09 | 4            |

**Business Insight:** Monthly order analysis showed fluctuations in customer demand over time, helping identify potential seasonal trends and business growth patterns.

## Regional Analysis

### Query 7: State-wise Revenue Analysis

**Objective:** Identify the regions contributing the highest sales revenue.

**SQL Concepts Used:** Multiple JOINS, GROUP BY, SUM, ORDER BY

**SQL Query:**

SELECT

```

c.customer_state,

ROUND(SUM(oi.price),2) AS revenue

FROM customers c JOIN orders o

ON c.customer_id = o.customer_id

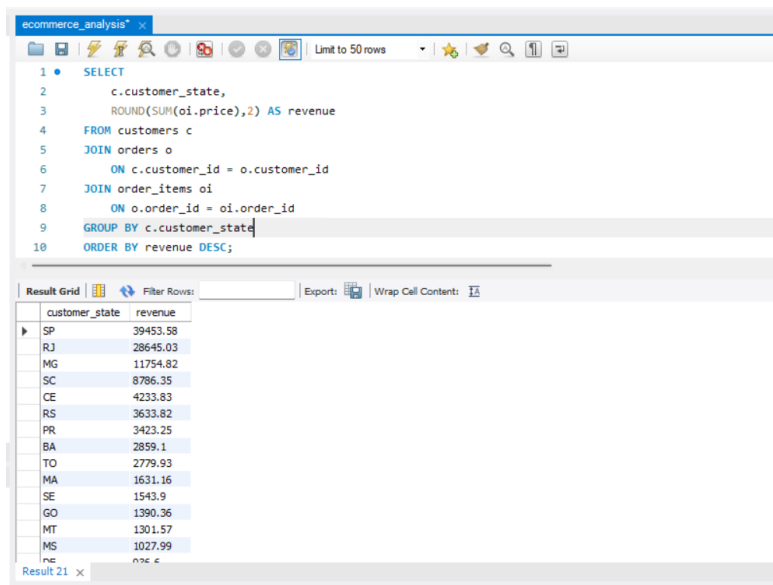
JOIN order_items oi

ON o.order_id = oi.order_id

GROUP BY c.customer_state ORDER BY revenue DESC;

```

### Output:



The screenshot shows a SQL query editor window titled "ecommerce\_analysis". The query is as follows:

```

1 SELECT
2   c.customer_state,
3   ROUND(SUM(oi.price),2) AS revenue
4 FROM customers c
5 JOIN orders o
6   ON c.customer_id = o.customer_id
7 JOIN order_items oi
8   ON o.order_id = oi.order_id
9 GROUP BY c.customer_state
10 ORDER BY revenue DESC;

```

Below the query, the "Result Grid" shows the output of the query. The table has two columns: "customer\_state" and "revenue". The results are sorted by revenue in descending order.

| customer_state | revenue  |
|----------------|----------|
| SP             | 39453.58 |
| RJ             | 28645.03 |
| MG             | 11754.82 |
| SC             | 8786.35  |
| CE             | 4233.83  |
| RS             | 3633.82  |
| PR             | 3423.25  |
| BA             | 2859.1   |
| TO             | 2779.93  |
| MA             | 1631.16  |
| SE             | 1543.9   |
| GO             | 1390.36  |
| MT             | 1301.57  |
| MS             | 1027.99  |
| PA             | 734.6    |

**Business Insight:** Revenue was concentrated in a few regions, with certain states generating significantly higher sales than others. This insight can help businesses prioritize marketing efforts and resource allocation in high-performing regions

### Query 8: Order Status Distribution

**Objective:** Analyze order fulfillment performance and identify the distribution of different order statuses.

**SQL Concepts:** GROUP BY, COUNT, Subquery, Percentage Calculation, ORDER BY

### SQL Query:

```

SELECT

order_status,

COUNT(*) AS total_orders,

ROUND(

COUNT(*) * 100.0 /

```



```

        (SELECT COUNT(*) FROM orders),

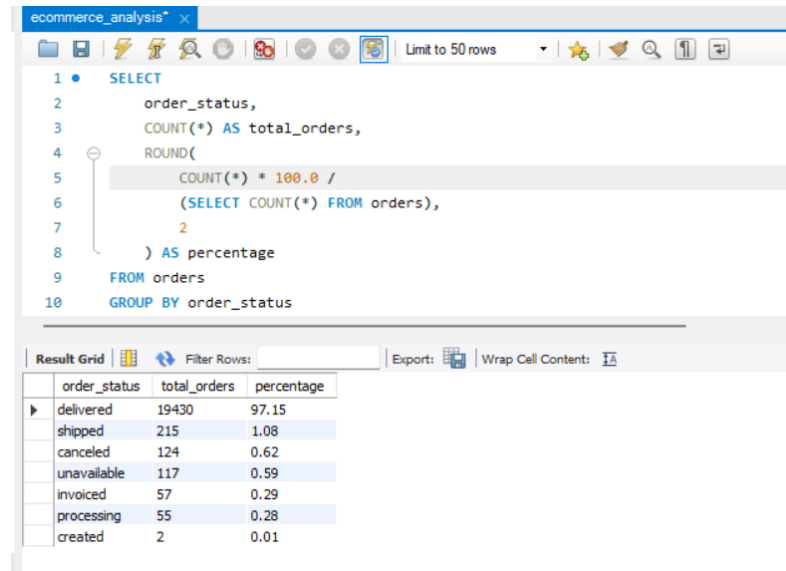
        2

    ) AS percentage

FROM orders GROUP BY order_status ORDER BY total_orders DESC;

```

### Output:



The screenshot shows a SQL query editor with the following query:

```

1 SELECT
2     order_status,
3     COUNT(*) AS total_orders,
4     ROUND(
5         COUNT(*) * 100.0 /
6         (SELECT COUNT(*) FROM orders),
7         2
8     ) AS percentage
9 FROM orders
10 GROUP BY order_status

```

Below the query, the 'Result Grid' shows the following data:

| order_status | total_orders | percentage |
|--------------|--------------|------------|
| delivered    | 19430        | 97.15      |
| shipped      | 215          | 1.08       |
| canceled     | 124          | 0.62       |
| unavailable  | 117          | 0.59       |
| invoiced     | 57           | 0.29       |
| processing   | 55           | 0.28       |
| created      | 2            | 0.01       |

**Business Insight:** The analysis revealed that the majority of orders were successfully delivered, while only a small percentage were cancelled, shipped, or in other processing stages. This indicates strong operational efficiency and effective order fulfillment performance.

### Query 9: State-wise Revenue Contribution Analysis

**Objective:** Measure each region's contribution to overall revenue and identify top-performing locations

**Business Question Answered:** The top revenue-generating regions contributed a substantial share of total revenue, indicating that sales performance is concentrated in specific geographical areas.

**SQL Concepts Used:** CTE (WITH), INNER JOIN ,GROUP BY, SUM, Subquery, Percentage Calculation, ORDER BY

### SQL Query:

```

WITH rev AS (

    SELECT

        c.customer_state,

        ROUND(SUM(oi.price),2) AS revenue

    FROM customers c

```

```

JOIN orders o

    ON c.customer_id = o.customer_id

JOIN order_items oi

    ON o.order_id = oi.order_id

GROUP BY c.customer_state
)

SELECT

    customer_state,

    revenue,

    ROUND(

        revenue * 100 /

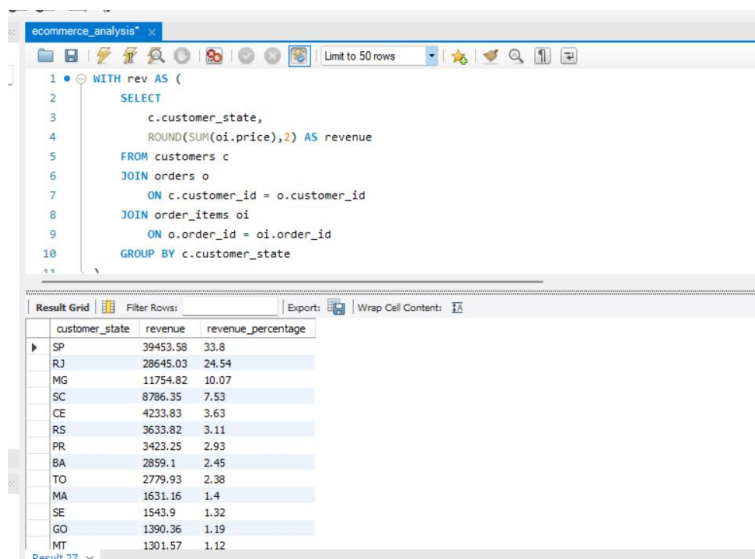
        (SELECT SUM(revenue) FROM rev),

        2

    ) AS revenue_percentage FROM rev ORDER BY revenue DESC;

```

### Output:



The screenshot shows a SQL query editor with a query window and a results window. The query window contains the following SQL code:

```

WITH rev AS (
    SELECT
        c.customer_state,
        ROUND(SUM(oi.price), 2) AS revenue
    FROM customers c
    JOIN orders o
        ON c.customer_id = o.customer_id
    JOIN order_items oi
        ON o.order_id = oi.order_id
    GROUP BY c.customer_state
)

SELECT
    customer_state,
    revenue,
    ROUND(
        revenue * 100 /
        (SELECT SUM(revenue) FROM rev),
        2
    ) AS revenue_percentage
FROM rev
ORDER BY revenue DESC;

```

The results window displays a table with 3 columns: customer\_state, revenue, and revenue\_percentage. The table contains 15 rows of data, sorted by revenue in descending order.

| customer_state | revenue  | revenue_percentage |
|----------------|----------|--------------------|
| SP             | 39453.58 | 33.8               |
| RJ             | 28645.03 | 24.54              |
| MG             | 11754.82 | 10.07              |
| SC             | 8786.35  | 7.53               |
| CE             | 4233.83  | 3.63               |
| RS             | 3633.82  | 3.11               |
| PR             | 3423.25  | 2.93               |
| BA             | 2859.1   | 2.45               |
| TO             | 2779.93  | 2.38               |
| MA             | 1631.16  | 1.4                |
| SE             | 1543.9   | 1.32               |
| GO             | 1390.36  | 1.19               |
| MT             | 1301.57  | 1.12               |

**Business Insight:** The majority of orders were successfully delivered, indicating strong order fulfillment performance and operational efficiency.

### Query 10: Order Status Distribution Analysis

**Objective:** Evaluate the percentage distribution of different order statuses.

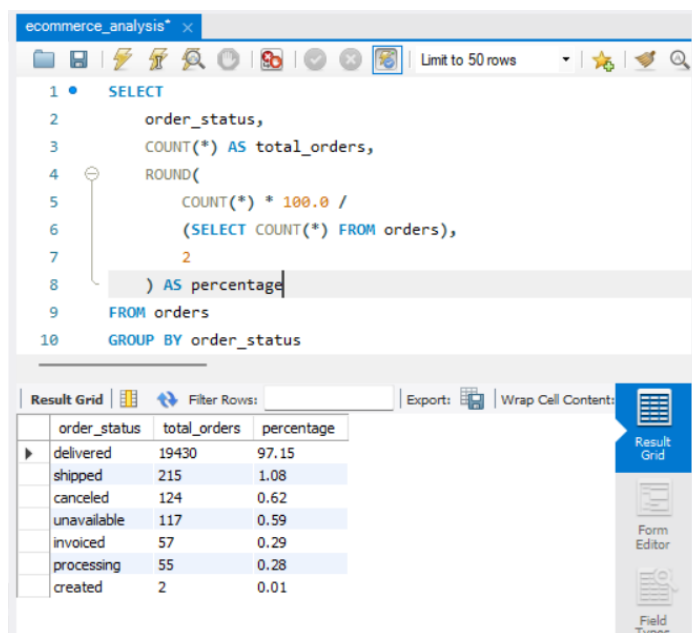
**Business Question Answered:** What percentage of orders were delivered, cancelled, shipped, or in other statuses?

**SQL Concepts Used:** GROUP BY,COUNT, Subquery, Percentage Calculation  
ORDER BY

### SQL Query:

```
SELECT  
  
    order_status,  
  
    COUNT(*) AS total_orders,  
  
    ROUND(  
  
        COUNT(*) * 100.0 /  
  
        (SELECT COUNT(*) FROM orders),  
  
        2  
  
    ) AS percentage FROM orders GROUP BY order_status ORDER BY total_orders DESC;
```

### Output:



The screenshot shows a SQL query editor window titled 'ecommerce\_analysis'. The query is as follows:

```
1 SELECT  
2     order_status,  
3     COUNT(*) AS total_orders,  
4     ROUND(  
5         COUNT(*) * 100.0 /  
6         (SELECT COUNT(*) FROM orders),  
7         2  
8     ) AS percentage  
9 FROM orders  
10 GROUP BY order_status
```

Below the query editor, the 'Result Grid' is displayed, showing the output of the query. The table has three columns: 'order\_status', 'total\_orders', and 'percentage'. The data is sorted by 'total\_orders' in descending order.

| order_status | total_orders | percentage |
|--------------|--------------|------------|
| delivered    | 19430        | 97.15      |
| shipped      | 215          | 1.08       |
| canceled     | 124          | 0.62       |
| unavailable  | 117          | 0.59       |
| invoiced     | 57           | 0.29       |
| processing   | 55           | 0.28       |
| created      | 2            | 0.01       |

**Business Insight:** More than 97% of orders were successfully delivered, demonstrating effective order management and customer fulfillment processes.

## Dashboard Development

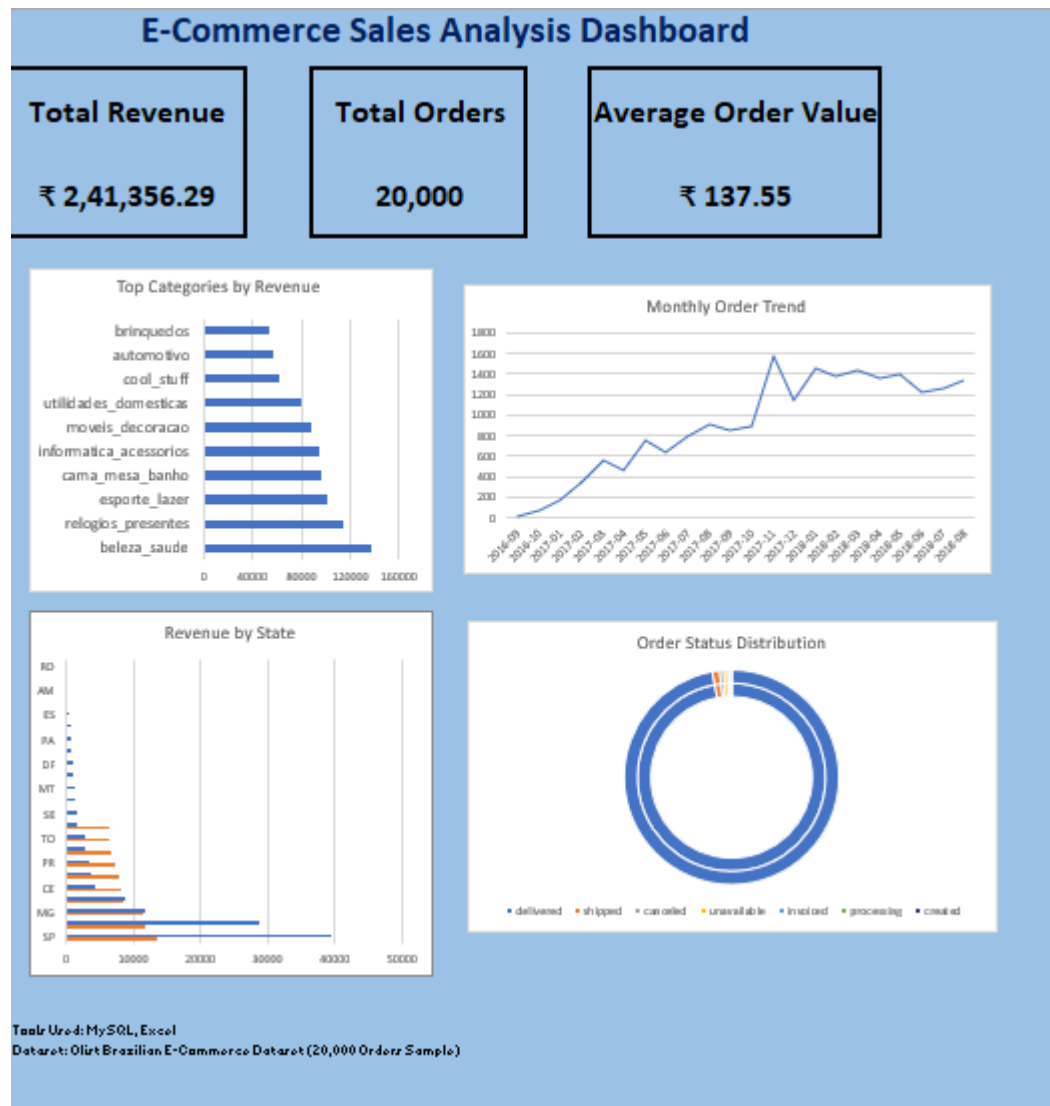
### Dashboard Overview

An interactive Excel dashboard was developed using the SQL analysis outputs to visualize key business metrics and performance indicators.

## Dashboard Components

- Total Revenue
- Total Orders
- Average Order Value
- Top Categories by Revenue
- Monthly Order Trend
- Regional Revenue Analysis
- Order Status Distribution

## Dashboard Screenshot



## Key Findings

1. The analysis generated a total revenue of **241,356.29** from the sampled e-commerce transactions, indicating substantial sales activity within the dataset.

2. Product-level revenue analysis revealed that a small group of products generated significantly higher revenue than the rest, with the top-performing product generating approximately **13,440** in sales revenue.
3. Category-wise analysis showed that revenue is concentrated in a limited number of product categories, suggesting that a few categories are the primary drivers of business performance.
4. Monthly order trend analysis identified variations in order volume over time, indicating changing customer demand and possible seasonal purchasing behavior.
5. State-wise revenue analysis showed that sales are not evenly distributed across regions, with certain states contributing a significantly larger share of overall revenue.
6. Revenue contribution analysis further confirmed regional concentration, highlighting the importance of focusing marketing and operational efforts on high-performing locations.
7. Order status analysis indicated that the majority of orders were successfully processed and delivered, reflecting effective fulfillment and operational performance.
8. The project successfully integrated multiple tables (customers, orders, order\_items, and products) to generate business insights using joins, aggregations, CTEs, subqueries, and date-based analysis.

## Conclusion

This project analyzed e-commerce sales data using SQL to evaluate revenue performance, product effectiveness, category contribution, regional sales distribution, monthly demand trends, and order fulfillment metrics. Through the use of SQL techniques such as joins, aggregations, CTEs, subqueries, and date functions, meaningful business insights were extracted from transactional data.

The analysis identified key revenue-generating products and categories, highlighted regional sales patterns, and assessed operational performance through order status evaluation. These insights can support data-driven decision-making in areas such as inventory planning, marketing strategy, regional expansion, and customer experience improvement.

Overall, this project demonstrates practical Data Analyst skills in data exploration, SQL-based business analysis, KPI development, and insight generation using a real-world e-commerce dataset.

The project demonstrates end-to-end data analysis skills, from data exploration and SQL querying to dashboard development and business insight generation.

## **Future Enhancements**

- Develop an interactive Power BI dashboard.
- Perform customer segmentation analysis.
- Implement sales forecasting models.
- Analyze customer retention and repeat purchase behavior.
- Automate reporting workflows using SQL and Power BI.

## **Prepared By:**

Abhineeth K

## **Tools:**

MySQL Workbench | Microsoft Excel